Kubernetes Notes

DevOps Assignment — Group 08

1. Introduction to Kubernetes

What is Kubernetes?

Kubernetes is an open-source container orchestration platform used to automate the deployment, scaling, management, and networking of containerized applications. It is commonly abbreviated as K8s.

Why Kubernetes?

Kubernetes helps deploy applications, manage containers, scale applications, restart failed containers, load balance traffic, manage updates, provide networking, and manage storage.

Container Orchestration

Container orchestration means automatically managing multiple containers. For example, Kubernetes can manage frontend, backend, and database containers and maintain the required number of running instances.

Kubernetes vs Docker

Docker is mainly used to create and run containers. Kubernetes is used to manage and orchestrate containers at scale.

Main Features

• Automatic scaling
• Self-healing
• Load balancing
• Rolling updates
• Service discovery

2. Kubernetes Architecture

Cluster

A Kubernetes cluster is a collection of machines called nodes that work together to run containerized applications. The major parts are the Control Plane and Worker Nodes.

Control Plane

The Control Plane manages the Kubernetes cluster. Its major components are API Server, Scheduler, Controller Manager, and etcd.

API Server

```
The Kubernetes API Server is the main communication point of the cluster. kubectl communicates with the API Serv
```

Scheduler

The Scheduler decides which worker node should run a Pod by considering available resources and scheduling requirements.

Controller Manager

Controllers continuously monitor the cluster and work to make the desired state equal to the actual state.

etcd

etcd is a distributed key-value store used by Kubernetes to store important cluster state and configuration information.

3. Kubernetes Components

Worker Node

A Worker Node is a machine where applications actually run. It generally contains kubelet, kube-proxy, a container runtime, and Pods.

kubelet

The kubelet is an agent running on each worker node. It communicates with the API Server, starts containers, monitors Pods, and reports their status.

kube-proxy

kube-proxy manages network communication for Services and helps route traffic to appropriate Pods.

Container Runtime

The container runtime is responsible for running containers. Examples include containerd and CRI-O.

4. Pods

What is a Pod?

A Pod is the smallest deployable unit in Kubernetes. A Pod usually contains one container, but it can contain multiple closely related containers. Containers in the same Pod share networking and can communicate using localhost.

Pod YAML Example

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
```

Commands

```
kubectl apply -f pod.yaml
kubectl get pods
kubectl describe pod nginx-pod
kubectl delete pod nginx-pod
```

5. ReplicaSet and Deployment

ReplicaSet

A ReplicaSet ensures that a specified number of Pod replicas are running. If a Pod fails, the ReplicaSet creates a replacement to maintain the desired number.

Deployment

A Deployment provides declarative updates for Pods and ReplicaSets. It supports scaling, rolling updates, rollbacks, and replica management.

Deployment YAML Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Scaling

```
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get pods
kubectl scale deployment nginx-deployment --replicas=5
```

6. Kubernetes Services

What is a Service?

A Kubernetes Service provides a stable network endpoint for accessing a group of Pods. Since Pod IP addresses can change, Services provide stable access.

Types of Services

1. ClusterIP — default type, accessible inside the cluster.
2. NodePort — exposes an application through a port on a node.
3. LoadBalancer — commonly used in cloud environments to expose an external load balancer.

Service YAML Example

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

7. Namespaces

What is a Namespace?

A Namespace logically divides resources inside a Kubernetes cluster. It is useful for organizing resources, separating environments, resource management, and access control.

Commands

```
kubectl get namespaces
kubectl create namespace development
kubectl get pods -n development
```

8. ConfigMaps and Secrets

ConfigMap

A ConfigMap stores non-sensitive configuration data such as application names, database hosts, environment values, and other configuration settings.

ConfigMap Example

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  APP_NAME: myapp
```

Secret

A Secret is used to store sensitive information such as passwords, API keys, tokens, and credentials. Kubernetes Secrets should still be handled carefully and are not automatically a replacement for a dedicated secrets manager.

9. Important Kubernetes Commands

Cluster

```
kubectl cluster-info
kubectl version
```

Nodes

```
kubectl get nodes
kubectl describe node &lt;node-name&gt;
```

Pods

```
kubectl get pods
kubectl get pods --all-namespaces
kubectl describe pod &lt;pod-name&gt;
```

Deployments and Services

```
kubectl get deployments
kubectl describe deployment &lt;deployment-name&gt;
kubectl get services
kubectl get svc
```

Logs and Exec

```
kubectl logs &lt;pod-name&gt;
kubectl exec -it &lt;pod-name&gt; -- /bin/bash
```

YAML and Resources

```
kubectl apply -f file.yaml
kubectl delete -f file.yaml
kubectl get all
```

10. Advantages and Use Cases

Advantages

• Scalability
• Self-healing
• Load balancing
• Automated deployment
• Rolling updates
• Portability across local, cloud, and hybrid environments

Use Cases

Kubernetes is commonly used for microservices, web applications, cloud-native applications, continuous deployment, large-scale applications, and DevOps automation.

Basic Kubernetes Flow

```
User → kubectl → API Server → Scheduler → Worker Node → Pod → Container
```

Reference

Kubernetes Official Documentation: https://kubernetes.io/docs/